



Dr.PathFinder: hybrid fuzzing with deep reinforcement concolic execution toward deeper path-first search

Seungho Jeon¹ · Jongsub Moon¹

Received: 19 April 2021 / Accepted: 30 January 2022 / Published online: 25 February 2022
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2022

Abstract

Fuzzing is an effective approach to discover bugs in programs, especially memory corruption bugs, using randomly generated test cases. However, without prior knowledge of the target program, the fuzzer can generate only a limited number of test cases because of sanity checks. To solve this problem, recent studies have proposed hybrid fuzzers that observe the context of a target program using symbolic execution; these fuzzers generate test cases to bypass the sanity check. While hybrid fuzzers explore “deep” bugs in the target program, they generate many ineffective test cases. In this paper, we propose a concolic execution algorithm that combines deep reinforcement learning with a hybrid fuzzing solution, Dr.PathFinder. When the reinforcement learning agent encounters a branch during concolic execution, it evaluates the state and determines the search path. In this process, “shallow” paths are pruned, and “deep” paths are searched first. This reduces unnecessary exploration, allowing the efficient memory usage and alleviating the state explosion problem. In experiments with the CB-multios dataset for deep bug cases, Dr.PathFinder discovered approximately five times more bugs than AFL and two times more than Driller-AFL. In addition to finding more bugs, Dr.PathFinder generated 19 times fewer test cases and used at least 2% less memory than Driller-AFL. While it performed well in finding bugs located in deep paths, Dr.PathFinder had limitation to find bugs located at shallow paths, which we discussed.

Keywords Fuzzing · Symbolic execution · Concolic execution · Reinforcement learning · Deep Q-network

1 Introduction

Fuzzing is a popular automated software testing method that randomly generates test cases to discover flaws in a program [1, 9, 38, 56, 57]. However, because programs have their own input formats, fuzzing achieves limited test coverage if the fuzzer creates test cases without prior knowledge of the program. Therefore, a coverage-based feedback fuzzer (gray-box fuzzer) that intensively utilizes test cases that trigger new coverage has recently been studied. American fuzzy loop (AFL) [57] is a landmark application of coverage-guided fuzzers. Although AFL has

successfully identified many bugs, complicated path constraints that arise from strict sanity checks such as magic bytes or checksums limit its ability to identify “deep” bugs.

Recently, hybrid fuzzers have been proposed to increase test coverage using dynamic taint analysis [33, 37] and symbolic execution [7, 10] to locate bugs deeper in the program. These techniques [41] observe the context of the running program directly or indirectly and induce the fuzzer to generate a test case that can bypass sanity checking. Dynamic taint analysis tracks all instructions of the program affected by the input value and collects values involved in the conditional branch; VUzzer [39] and Angora [9] are examples of this method. Symbolic execution uses symbolic inputs to simulate all possible paths. In this process, the symbolic executor collects the branch condition path constraints that enable each path to be executed. When the execution is completed, it invokes a satisfiability modulo theory solver [3, 11] to derive symbolic inputs that make each path feasible. The generated

✉ Jongsub Moon
jsmoon@korea.ac.kr
Seungho Jeon
ohgnu90@korea.ac.kr

¹ Division of Information Security, Graduate School of Information Security, Korea University, Seoul, Republic of Korea

input serves as a test case for the fuzzer. Driller [47], Mayhem [8], and QSYM [55] use symbolic execution to increase the test coverage of the fuzzer.

Although dynamic taint analysis and symbolic execution successfully improve performance, these approaches still have some drawbacks. For example, the former has an overhead because it traces too many instructions during analysis, and the latter traces all possible paths of a program, which results in a path explosion problem (i.e., state explosion). If the semantics of the instruction are not implemented correctly, the simulation for the program may differ from the actual execution [24]. Most importantly, symbolic execution requires a considerable amount of time to invoke the SMT solver to resolve the path constraints of all traced execution paths, even if some that are not useful for extending the coverage of the fuzzer. Further, the fuzzer augmented with symbolic execution cannot utilize these unhelpful paths to search for “deep” bugs in the program. These techniques which augment the fuzzer share the common problem. Since the fuzzer searches bugs sequentially from the shallow paths, the search for deep bugs has a lower priority. Therefore, a method is needed to prioritize to search deep bugs that are relatively difficult to discover due to complex path constraints.

In this paper, we focus on exploring bugs located deep path in the program’s execution paths. To achieve this objective, a concolic execution algorithm combined with deep reinforcement learning and hybrid fuzzing solution, called Dr.PathFinder, is proposed to find deep bugs. Reinforcement learning is a research field in control theory and artificial intelligence, and deep learning [29]-based techniques have been actively employed in this field. A reinforcement learning agent performs actions and receives rewards for every state observed in the given environment. The agent is trained to maximize the sum of the accumulated rewards at the end of an episode. Reinforcement learning characteristics allow the agent to search for the optimal policy and control at the human level [36, 45, 46, 51]. Program analysis, including vulnerability discovery, is known as an undecidable problem due to its infinite state space [27]. Therefore, rather than tabular methods such as Q-Learning [53] or SARSA [40] for tracing all states, Dr.PathFinder approximates states using a deep neural network. Then, based on this approximation, Dr.PathFinder selects the branch where the path is expected to continue longer. For this objective, we propose a learning algorithm that allows the agent to search primarily for paths expected to be deeper during concolic execution. The trained agent selectively prunes branches that are not useful for improving coverage. This eventually reduces the search space of concolic execution by reasonably limiting the generation of inputs, thereby enabling the fuzzer to find deeper bugs. The contributions of this study are as follows:

- We formally define a learning algorithm for a deep reinforcement learning agent that allows concolic execution to first search for a deeper path.
- We present a deeper path-first search concolic execution algorithm using a reinforcement learning agent and a hybrid fuzzer called Dr.PathFinder.
- We show the bug discovery performance of Dr.PathFinder and the efficient resource utilization ability of deep reinforcement concolic execution using a benchmark dataset.

On the other hand, the proposed approach had some obvious limitations, which we discuss in more detail in Sect. 5.

- Unlike most fuzzers which roughly sequentially search from shallow to deep paths, Dr.PathFinder focuses on following quickly “deeper paths” from the beginning by design. Due to this, a tendency to miss bugs located at shallow paths is observed.
- The objective of our approach is not directly coupled with bug findings. It implies that to keep finding new bugs, sometimes non-optimal actions are required. So the concolic execution needs to keep the stochastic policy, making it difficult to converge to the optimal policy.

Despite the above limitations, Dr.PathFinder showed higher discovery performance than existing fuzzers for bugs located at deep paths.

The remainder of this paper is organized as follows. Section 2 presents the research on fuzzing and reinforcement learning. The concolic execution algorithm combined with reinforcement learning and hybrid fuzzing with the algorithm is presented in Sect. 3. In Sect. 4, we detail the comprehensive experimental results for the proposed hybrid fuzzing, and some limitations are discussed in Sect. 5. Finally, the conclusion is provided in Sect. 6.

2 Backgrounds and related work

Many studies have been conducted to overcome the limitations of existing coverage-guided fuzzing. This section introduces the recently presented coverage-guided fuzzers and hybrid fuzzers that employ augmenting techniques and describes the background knowledge of recent reinforcement learning.

2.1 Coverage-guided hybrid fuzzing approaches

Coverage-guided fuzzers were spotlighted in the field of automated vulnerability detection. They use feedback from the program for test cases without heavy static analysis of

the target program. In particular, AFL [57] and its variations successfully found a large number of bugs. As of September 20, 2021, the AFL had found more than 363 vulnerabilities in more than 160 programs¹. AFL inserts an instrument for each basic block while compiling or executing the source code. The instrument records the transition (coverage) of the basic block executed while the target program is processing the test case. When the fuzzer finds new coverage, it queues the test case and creates new test cases using a mutation algorithm. However, mutation scheduling of vanilla AFL is inefficient. MOPT [34] optimized the scheduler using a modified particle swarm algorithm [43]. Another approach to improve the mutation scheduler is by using the Markov model; AFL-fast [4] adjusted the number of mutations based on a simple Markov chain. Honggfuzz [18] was designed to perform fuzzing more efficiently using a strategy similar to that of AFL. This strategy had a significant advantage in measuring the quality of the test case compared to that of pure black-box fuzzing. Although AFL has proven its effectiveness, it has been shown to be ineffective in programs that employ sanity check routines. Recent fuzzing studies have presented methods to overcome this drawback; dynamic taint analysis and symbolic execution are commonly used to this end.

Dynamic taint analysis is a technique that analyzes the dynamic characteristics of a target program by either tracing all instructions or some interesting instructions of a running target. This technique has been used for software debugging [15, 35] or information leakage detection [14, 48]. Furthermore, it has been actively introduced for fuzzing. BuzzFuzz [16] used taint analysis to specify the part of the input value involved in the library or system calls of the target programs. VUzzer [39] used a lightweight static analysis before starting, and then carried out a dynamic taint analysis to collect operands for “cmp” and “lea” commands. In addition, it utilized the collected commands for a mutation to increase the possibility of passing through branches and prioritizing the test cases, triggering the paths that do not include error handling blocks. Angora [9] employed lightweight taint tracking to identify the input values involved in path constraints, and introduced a novel search strategy based on type-and-shape inference and gradient descent to solve path constraints without symbolic execution efficiently.

Symbolic execution is a static analysis technique that simulates program execution using symbolic inputs. This technique traces all the branches encountered while simulating the target program and collects the path constraints. Then, it assigns a concrete value to the symbolic input to

realize the path using the SMT solver at the end of each path. Driller [47] is invoked when the AFL cannot find any new coverage (“stuck”) and performs concolic testing [17, 42] using the test case generated by the fuzzer. It solves the condition that causes “stuck” and creates a new input value that navigates deeper paths. Finally, these paths are provided to the fuzzer. DigFuzz [58], which is a variation of Driller, causes the concolic execution to search for the least-visited branches by AFL using a Monte Carlo method. Mayhem [8] used a Driller-like search strategy; however, it used hybrid symbolic execution for more efficient memory usage. Hybrid symbolic execution starts with online execution and switches to offline symbolic execution (equal to concolic execution) when the memory usage approaches the upper limit.

Dynamic taint analysis and symbolic execution each have their own advantages. Studies have proposed combining both these techniques in fuzzing to create a synergy for finding bugs. TaintScope [52] used taint analysis to identify inputs related to security-sensitive operations, such as library calls or system calls. In this process, an incorrect checksum was fixed using symbolic execution so that the execution of the target was prematurely terminated by the checksum. Dowser [19] was proposed to find buffer overflows and underflows; it used a taint analysis to identify the bytes associated with the index of the target’s buffer from the input value, which made the target symbolic by changing only bytes into symbolic input.

2.2 Reinforcement learning

An agent in reinforcement-learning interacts with the environment. The agent can observe only one state, which provides partial or full information about the environment. Furthermore, it can perform actions considered optimal based on the policy in every state (but not always the optimal one due to a stochastic policy). The agent receives a reward for every action from the environment and transitions to a new state. The goal of reinforcement learning is to learn a policy that maximizes the expectation of the sum of rewards (return) collected by repeating these processes. If the environment is a white box, the agent can know in advance all possible states. Therefore, the agent can determine in advance the optimal action in each state and the state transition. This is called the Markov Decision Process (MDP). In MDP, a policy is defined based on a state or action-value function:

$$V_{\pi} = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) (r + \gamma V_{\pi}(s')) \quad (1)$$

$$Q_{\pi}(s,a) = \sum_{s',r} p(s',r|s,a) (r + \gamma V_{\pi}(s')) \quad (2)$$

¹ This number only counts vulnerabilities reported at <https://lcamtuf.coredump.cx/afl/>.

where V_π and Q_π are the state and action value function under a policy π , respectively. s , s' , and a denote the state at the current timestep, state at the next timestep, and action chosen by the agent in s following the policy π , respectively. Further, a' and r denote the action at s' and the reward for a at s , respectively. γ denotes a discount factor; therefore, it does not allow the return to diverge ($0 < \gamma < 1$). As shown in Eqs. (1) and (2), $V_\pi(s)$ and $Q_\pi(s, a)$ are defined recursively ($V_\pi(s) = \sum_a \pi(a|s)Q_\pi(s, a)$). That is, the state value is defined as the average of the possible actions in the current state s and the next state value according to each action. Similarly, the action value is defined as the average of the next state value by an arbitrary action a in the current state s . Since MDP knows all states and actions, it is easy to learn the optimal policy through policy iteration or value iteration algorithm [49].

However, in the real world, in many cases, it is not possible to fully cover the environment. Therefore, the agent should evaluate the value of the state or action based on its own experience. Numerous algorithms have been proposed for solving this problem. Q-learning [53], which is an off-policy algorithm, is one of the most significant breakthroughs in reinforcement learning. Off-policy implies that the policy to secure the diversity of behavior and the policy to be learned are separated in the learning process. The formula for the policy update in Q-learning is defined as

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)) \quad (3)$$

In the context of reinforcement learning, “ $r + \gamma \max_{a'} Q(s', a')$ ” is called a target. The term “target” in this Q-learning is different from that in the target program used in the fuzzing. Thus, in Q-learning, the gradient is the difference between the predicted value of a at s and the target, and π gradually learns to reduce this difference. The resulting π^* minimizes the difference.

Q-learning has been successfully used in many applications [2, 54]; however, it has some limitations because it employs the tabular method. As the size of the state space and the dynamics of the environment increase, the agent cannot maintain $Q(s, a)$ for all observed states and actions. Therefore, the value function must be approximated. Recently, deep learning has been used to approximate $Q(s, a)$ [29]; deep Q-network (DQN) is one successful approximation for the value function, and in a DQN, the action-value function is replaced by a deep learning model. The objective function $\mathcal{L}$ is defined as follows:

$$\mathcal{L} = (y - Q_\theta(s, a))^2$$

$$y = \begin{cases} r & \text{for terminal state } s' \\ r + \gamma \max_{a'} Q_\theta(s', a') & \text{for non-terminal state } s' \end{cases} \quad (4)$$

where $Q_\theta(s, a)$ denotes the value of action a in s predicted by the deep learning model parameterized by θ . In DQN, $Q_\theta(s, a)$ is learned to minimize $\mathcal{L}$ using the gradient descent method [5]. In addition, DQN introduced two engineering approaches—experience replay [32] and network separation—to stabilize learning. Experience replay does not use immediate observations for learning the Q-network; however, it stores the immediate observations in replay memory and trains the Q-network using past observations. The network separation builds two Q-networks with the same parameters at the beginning of the episode: one to calculate the target value (target network) and the other to predict the action value (prediction network). In one episode, only the prediction network is learned, and the parameter θ' of the target network is periodically updated with parameter θ of the prediction network. These techniques dramatically stabilized the learning progress of the Q-network and prevented overfitting.

3 Hybrid fuzzing framework with deep reinforcement concolic execution

In this section, we propose Dr.PathFinder, a hybrid fuzzer based on deep reinforcement learning. Unlike traditional concolic execution, our approach adaptively selects a path to be searched along with the target program. Rather than relying on a human-crafted heuristic for selecting the path, the proposed concolic execution traces branches that can lead to a deeper path based on experience. Reinforcement learning algorithms such as DQN repeatedly execute the same or similar experiences. The experience provides information that agents can use to learn the best policy to solve problems in the given environment. The optimal policy is an action selection strategy for the agent to optimally solve the problem. Section 3.1 provides an overview of the entire framework of the proposed method. In Sect. 3.2, we formally describe a DQN agent tailored to concolic execution. In Sect. 3.3, we present a concolic execution algorithm using the agent.

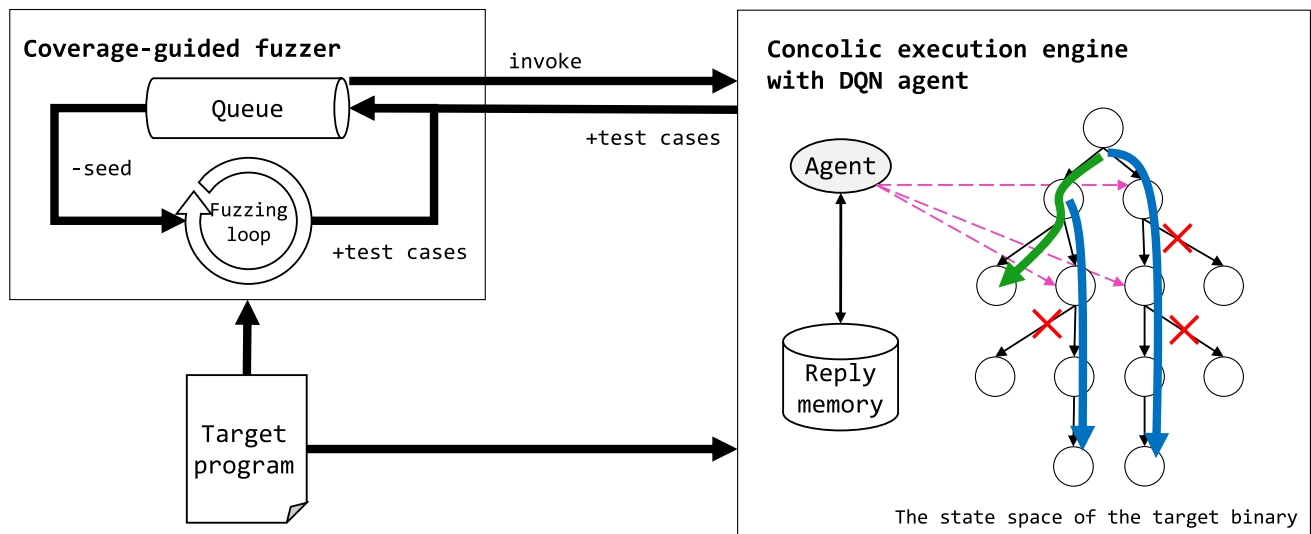


Fig. 1 Overview of hybrid fuzzer framework with deep reinforcement concolic execution

3.1 Overview of proposed method

Figure 1 shows the proposed hybrid fuzzing framework. The mechanism of the entire framework and deep reinforcement concolic execution is provided as follows:

1. Setup fuzzer: The first coverage-guided fuzzer takes both a target program and a queue with simple test cases as arguments.
2. Fuzzing loop: The fuzzer takes a test case from the queue and uses it as a seed for the mutation algorithm to generate a new test case. The target program is executed with this test case as the input.
3. Coverage feedback: When the fuzzer finds a new basic block transition (coverage) or crashes while the target program is running, the fuzzer considers the test case as interesting and queues the test case.
4. Concolic testing: If no new coverage is found after repeating the fuzzing loop step, the fuzzer invokes the deep reinforcement concolic execution engine. This engine first obtains test cases from the queue and runs the target program concretely (green path). The engine then negates the path constraints of the branch encountered during concrete execution and starts symbolic execution. When the DQN agent encounters a conditional statement, it leads the symbolic execution to trace the branch where a deeper path is expected (magenta dashed arrows). After repeating the branch selection and tracing the paths with the DQN agent (blue colored paths), the SMT solver resolves the collected path constraints and generates new test cases to realize the paths. The generated test cases are provided to the queue and used in the fuzzing-loop step.

In Dr.PathFinder, we used AFL [57] as a base model for the coverage-guided fuzzer. In the remainder of this section, we focus on the concolic execution engine with a DQN agent, which is the core of our insight. In Sect. 3.2, the DQN agent for a deeper path-first search is formally defined. In Sect. 3.3, the concolic execution algorithm using this agent is described.

3.2 Formal description of DQN agent for deeper path-first search

Concolic execution, coupled with fuzzing, is considered an effective method for program testing. While the method effectively reduces the search space using concolic execution, it is difficult for the fuzzer to focus on bugs located on deeper paths as a result of unnecessary searching for shallow paths that hardly lead to a bug. To solve this problem, we propose a DQN adapted to concolic execution. Whenever the agent encounters a branch during concolic execution for the target program, it evaluates the basic block (state s), selects the branch that is likely to lead deeper (action a), and receives a reward r . Longer paths lead to larger returns. Experiences that have been collected are used as input. The properly trained agent performs the optimal action when selecting a branch for exploration. In this section, a DQN tailored for a deeper path-first search is formally defined.

3.2.1 State

In the context of reinforcement learning, a state is a partial observation of the environment. A program's state can be defined in various ways. For an executed program, we may obtain different types of information, such as memory

snapshots and processor context. However, such information may change with each execution because of factors such as the operating system, environment variables, and random number, even if the same code is executed. Therefore, we use the instructions to consistently represent the program's state during concolic testing of the target program instead of the above dynamic information.

We define the state s_t of the program at time step t with the basic block BB_t observed by the agent. For programs that dynamically generate executable instructions, the basic block is a more suitable material than the runtime information for defining a program's state. BB_t denotes a series of instructions included in the basic block as follows:

$$BB_t = \{inst_{t,1}, inst_{t,2}, \dots, inst_{t,|BB_t|}\} \quad (5)$$

where $|BB_t|$ denotes the number of instructions in the basic block BB_t . $inst_{t,i}$ denotes the i -th ($0 < i \leq |BB_t|$) instruction included in BB_t .

Modern symbolic execution tools use intermediate representation (IR) [6, 28, 37] to support multiple platforms. We first lift $inst_{t,i}$ to IR, to employ the agent in a cross-platform environment as

$$BB_t^{ir} = \{inst_{t,1}^{ir}, inst_{t,2}^{ir}, \dots, inst_{t,|BB_t^{ir}|}^{ir}\} \quad (6)$$

where BB_t^{ir} denotes the basic block lifted to IR observed by the agent at time step t , $|BB_t^{ir}|$ represents the number of IR instructions included in BB_t^{ir} , and $inst_{t,j}^{ir}$ denotes the j -th ($0 < j \leq |BB_t^{ir}|$) instruction included in BB_t^{ir} . Hereafter, the basic block and instructions in this paper refer to IR-lifted basic blocks and instructions, respectively.

Basic blocks defined by byte-level machine codes or string-based mnemonics are not suitable as inputs for the agent. Therefore, instructions or further basic blocks should first be quantized into d -dimensional vectors using the preprocessing function $\phi: inst_{t,j}^{ir} \rightarrow \mathbb{R}^d$. The basic block at the quantized time step t is used as the program state s_t at time step t , and it is defined as

$$s_t = \{\phi(inst_{t,1}^{ir}), \phi(inst_{t,2}^{ir}), \dots, \phi(inst_{t,|BB_t^{ir}|}^{ir})\} \quad (7)$$

3.2.2 Action

Concolic execution handles two branches resulting from the conditional statements during execution. A true branch is a transition that occurs when a conditional statement is satisfied, and the opposite is true for a false branch. The DQN agent evaluates the state s_t of the program at time step t when it encounters the branch, selects a branch (action a_t) where a longer path is expected, and observes the next state s_{t+1} . Branch selection follows policy π , which may be probabilistic or deterministic. DQN, an off-policy algorithm, uses both types of policies to train an agent. In this study, we use ϵ -

soft as a probabilistic policy ($a_t \sim \pi(a|s_t)$) and max operation for a deterministic policy ($a_t \leftarrow \max_a Q_\theta(s_t, a)$). The ϵ -soft policy is used to secure the diversity of experiences through exploitation and exploration when the agent selects an action. The maximum greedy policy is used to calculate the target when learning a DQN, where exploitation and exploration imply selecting an action with the largest value of $Q_\theta(s, a)$ and selecting an action randomly, respectively. To define such a policy, the set of actions $\mathcal{A}$ ($a_t \in \mathcal{A}$) that the agent can take must be defined. We use action set $\mathcal{A}$, which consists of three actions:

$$\mathcal{A} = \{tb, tt, tf\} \quad (8)$$

where tb stands for “taking both branches” and keeps all branches; tt is short for “taking the true branch,” which prunes the false branch and keeps only the true branch; and tf stands for “taking the false branch,” which prunes the true branch and keeps only the false branch. There was no action to prune every branch. Considering that this behavior is sufficiently simple, if the agent discards all branches, the paths will no longer proceed, and therefore, the search capability of concolic execution may be unnecessarily limited.

This definition of the action set is different from the original DQN [36, 51]. Typically, in ordinal reinforcement learning algorithms, including the original DQN, the agent takes an action a_t in state s_t , and transitions to only one next state s_{t+1} . However, in our action set definition, we observe up to two next states because of action tb , which selects all branches. Therefore, the transfer function ψ for concolic execution using the DQN agent in the branch is defined as:

$$\psi: s_t, a_t \rightarrow \Sigma_{t+1} \quad (9)$$

where $\Sigma_{t+1} \in \{\{s_{t+1}^\top\}, \{s_{t+1}^\perp\}, \{s_{t+1}^\top, s_{t+1}^\perp\}\}$ denotes the set of next states observed by taking a_t in s_t ; $s_{t+1}^\top$ denotes the next state following the true branch; and $s_{t+1}^\perp$ denotes the next state following the false branch.

3.2.3 Reward

The DQN agent takes an action at time step t and receives and accumulates the reward r_t . Therefore, the sum of the accumulated rewards (return) either increases or decreases as the execution path increases, with more branches traversed. Based on this experience, the agent evaluates all possible actions in s_t and selects action a_t , which is expected to achieve the largest return. In principle, the reward r_t can be any real value. For instance, if the agent can observe an immediate reward for an action, such as a game score, one can directly use the score as a reward. However, in concolic execution, an explicit reward cannot

be acquired for an action. Therefore, we set the reward to a positive value to equally evaluate each branch's value. Accordingly, the more the agent observes the state-action pair (s_t, a_t) during an episode, the higher the increase in accumulated rewards. This is potentially reflected in $Q_\theta(s_t, a_t)$, thereby allowing the agent to choose the optimal action, namely a longer path. If the reward is set to a negative value, the accumulated reward decreases. Eventually, the agent selects a short path according to the *maximum* greedy policy.

Another important aspect is the definition of the following states: Typically, the next state can be specified as the only one after the agent takes an action. However, following the definition in Sect. 3.2.2, the agent observes as many next states as the number of traced branches, both true and false. This definition for the next states enables the agent to receive the set of immediate rewards $P_t \in \{\{r_t^\top\}, \{r_t^\perp\}, \{r_t^\top, r_t^\perp\}\}$ for the number of traced branches. $r_t^\top, r_t^\perp \in \mathbb{R}$ are single-reward values at time step t when the true branch and the false branch are selected, respectively. The size of P_t is the same as that of Σ_{t+1} .

3.3 Deep reinforcement concolic execution for deeper path-first search

In this section, we present a concolic execution algorithm for a deeper path-first search based on the definitions provided in the previous section. Although this algorithm is generalized so that it is not restricted to the form of the Q-function, we present the proposed Q-network architecture in Sect. 3.3.2.

3.3.1 Deep reinforcement concolic execution algorithm

Algorithm 1 is similar to the algorithm proposed by Mnih et al. [36, 51]; however, the definition of the target value y_k in the algorithm is slightly different in our action set definition. While tt and tf actions can be intuitively defined as the sum of immediate reward and discounted returns from s_{k+1} , action tb does not. We used the average of the action values between tt and tf to represent the expectation of the action value for tb . It can be expressed uniformly as Eq. (10). Because of the tb action, y_k is defined as the average of the target values for all the following states in Σ_{k+1} . Calculating the average value of the target for the number of next states ensures that the action value of tb cannot be greater than $\max(Q_\theta(\text{tt},.), Q_\theta(\text{tf},.))$ because it is the average of $Q_\theta(\text{tt},.)$ and $Q_\theta(\text{tf},.)$. Since the policy is based on the action value and the value is less than or equal to $\max(Q_\theta(\text{tt},.), Q_\theta(\text{tf},.))$, the frequency of tb action selection cannot be higher than tt or tf . Thus, tb is not selected always.

$$y_k = \frac{1}{|\Sigma_{k+1}|} \sum_{s_{k+1} \in \Sigma_{k+1}} \Gamma_{s_{k+1}}$$

$$\Gamma_{s_{k+1}} = \begin{cases} r_k & \text{if } s_{k+1} \text{ is terminal state} \\ r_k + \gamma \max_{a'} Q_{\theta'}(f(s_{k+1}), a') & \text{otherwise} \end{cases} \quad (10)$$

Equation (10) shows how to achieve the target value, where θ' denotes the θ calculated several time steps ago, which is a parameter of the Q-network; γ denotes the discount factor ($0 < \gamma < 1$); $f(s_t)$ denotes a function that uses s_t as an input, thereby returning the concatenated state from time step $t - \tau$ to time step t :

$$f : s_t \rightarrow \{\phi(\text{inst}_{t-\tau,1}^{ir}), \dots, \phi(\text{inst}_{t,1}^{ir}), \dots, \phi(\text{inst}_{t,|BB_t^{ir}|}^{ir})\} \quad (11)$$

In addition, it allows the Q-network to capture the chronological feature of s_t with the same modality and with the definition of the state, as shown in Eq. (7).

Algorithm 1: Learning algorithm of the DQN agent tailored to the concolic execution.

```

initialize reply memory  $\mathcal{D}$  with capacity  $N$ ;
initialize action-value function  $Q_\theta$  with random weights;
ACTIVE  $\leftarrow \emptyset$ ;
foreach test-case  $\sigma$  in fuzzer's queue do
    start the concolic execution with  $\sigma$  and get a quantized initial state  $s_1$ ;
    ACTIVE  $\leftarrow \text{ACTIVE} \cup \{s_1\}$ ;
    repeat
        ACTIVE'  $\leftarrow \emptyset$ ;
        foreach  $s_t \in \text{ACTIVE}$  do
            do concolic execution for  $s_t$  and get a set of next states  $\Sigma_{t+1}$ ;
            if  $|\Sigma_{t+1}| > 1$  then
                 $a_t \leftarrow$  get  $a_t$  following  $\epsilon$ -soft policy with  $Q_\theta(f(s_t), a_t)$ ;
                if  $a_t = \text{tb}$  then
                    do nothing;
                else if  $a_t = \text{tt}$  then
                     $\Sigma_{t+1} \leftarrow \Sigma_{t+1} \setminus s_{t+1}^\perp$ ;
                else if  $a_t = \text{tf}$  then
                     $\Sigma_{t+1} \leftarrow \Sigma_{t+1} \setminus s_{t+1}^\top$ ;
                ACTIVE'  $\leftarrow \text{ACTIVE}' \cup \Sigma_{t+1}$ ;
                receive a reward set  $P_t$ ;
                 $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, P_t, \Sigma_{t+1})\}$ ;
                 $\{(s_k, a_k, P_k, \Sigma_{k+1})\}_{k=1}^m \sim \mathcal{D}$ ;
                 $\mathcal{L} \leftarrow \frac{1}{m} \sum_{k=1}^m (y_k - Q_\theta(f(s_k), a_k))^2$ ;
                 $\theta' \leftarrow \theta' - \eta \nabla \mathcal{L}$ ;
            end
        end
        ACTIVE  $\leftarrow \text{ACTIVE}'$ ;
    until ACTIVE is empty;
     $\theta \leftarrow \theta'$ ;
end

```

Algorithm 1 is invoked only when the fuzzer cannot expand the coverage. Concolic execution is conducted using test cases generated by the fuzzer as seeds. The DQN agent selects an action a_t following the ϵ -soft policy for every branch, starting from the initial state s_1 . If the selected action a_t is **tb**, all branches are maintained; if a_t is **tt**, the false branch is removed; if a_t is **tf**, the true branch is removed. In this way, Σ_{t+1} and its reward set P_t are received; the transition pair $(s_t, a_t, P_t, \Sigma_{t+1})$ is stored in the replay memory. Next, the objective of the DQN is calculated with randomly sampled m transition pairs $\{(s_k, a_k, P_k, \Sigma_{k+1})\}_{k=1}^m$ observed in past experiences from the replay memory. These transition pairs may contain experiences with both current and previous testing targets. The Q-network is trained with sampled transitions: the technique of learning the Q-network asynchronously using replay memory is called experience replay [32]. This effectively prevents the overfitting of Q-network learning. This process is repeated until the concolic execution ends for one seed. When the concolic execution is completed, θ' of the target network is updated to the parameter θ of the prediction network. As shown in the algorithm, the policy depends on the Q-network, which is iteratively updated. This means that the policy is merely fixed but is trained incrementally. This incremental learning leads the concolic execution to even smarter exploration of testing targets.

Once the search for all paths originating from the seed is complete, the concolic execution engine invokes the SMT solver to create test cases that realize these paths. As a result of our prior experiments, we observed that many branches were pruned despite not using the action that discards all branches, thereby limiting the search capability of concolic execution. To compensate for this limitation, test cases for s_{t+1} pruned by action **tt** or **tf** are generated stochastically in proportion to the depth of the search. The pruned s_{t+1} is no longer traced.

The DQN agent does not merely prune the branches with knowledge gained from a single program. Instead, it is based on the experience gained from many programs. Therefore, rather than selecting the optimal action for a single program, it guides the concolic execution of a deeper path by learning the execution patterns of general programs.

3.3.2 Deep Q-network architecture

To determine the action value in an infinite state space, the Q-function is designed with a deep neural network called a Q-network. The agent selects the optimal action a_t in s_t using the Q-network. Originally, Mnih et al. [36, 51] designed a Q-network using a convolutional neural network (CNN) to process visual inputs. However, in this

study, the state of the program is defined as the IR instructions in the basic block; thus, owing to the difference in the number of IR instructions contained in each basic block, a recurrent neural network (RNN) is adopted instead of the CNN.

Figure 2 shows the Q-network architecture used in this study. This model is composed of an RNN to handle the state s_t with variable lengths. In addition, a long short-term memory (LSTM) cell [21] is used to realize robustness against long/short-term dependency. The RNN is composed of multiple layers to secure sufficient network capacity, and a batch normalization [22] layer is placed between the RNN layers. Therefore, the distribution of activations can be stabilized and the gradients can be propagated well during training. This model predicts the value for each action in state s_t by adding a dense layer, batch-normalization layer, and finally, a dense layer at the output of the RNN at the last time step.

4 Evaluations of Dr.PathFinder

We implemented Dr.PathFinder using the algorithm presented in the last section. We conducted several experiments to evaluate its performance. First, we introduce the details of the implementation of Dr.PathFinder. Then, we present the several experimental results. In the experiments, AFL [57], AFL-fast [4], AFL-laf-intel [26], and Driller-AFL [47] were used for performance comparison. The technical strategies of each fuzzer are summarized in Table 1.

4.1 Implementation

We built Dr.PathFinder on top of Driller—a state-of-the-art concolic execution engine—and we internally used Angr [44], which is a symbolic execution engine. Since Angr uses VEX as an intermediate language for symbol

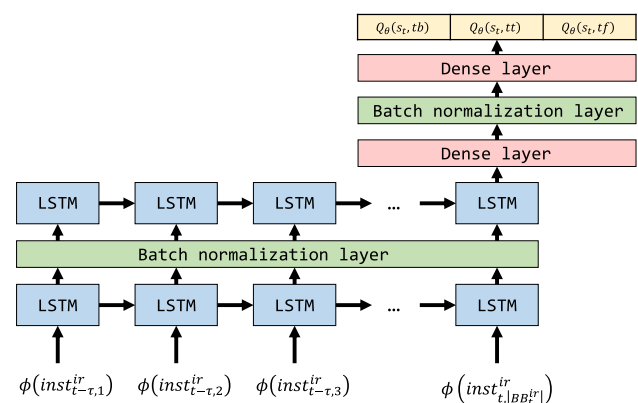


Fig. 2 Q-Network for DQN agent tailored for concolic execution

Table 1 Technical strategies of fuzzers

Fuzzers	Description
AFL	Mutational coverage-based fuzzing
AFL-fast	Mutational coverage-based fuzzing + Markov Model-based search
AFL-laf-intel	Mutational coverage-based fuzzing + compare instruction transformation
Driller-AFL	Mutational coverage-based fuzzing + concolic execution
Dr.PathFinder	Mutational coverage-based fuzzing + deeper path first concolic execution

execution, the lifting function of Angr is responsible for the ϕ function in Sect. 3.2.1. Then, we used the AFL 2.52b version as the fuzzer interacting with the concolic execution engine. Tensorflow-gpu 2.2.0, was used to implement the DQN agent, and Angr was slightly modified to obtain the state transition. In addition, Dr.PathFinder still followed the scheduling of the original Driller for concolic execution. That is, concolic execution is invoked when the `pending_favs` attribute in the AFL falls to 0. However, unlike Driller-AFL, which conducts a concolic execution once for all test cases generated by AFL, Dr.PathFinder allows test cases that have already been used as seeds to be stochastically used as seeds again. Further, it provides an opportunity to search in the pruned path through deep reinforcement concolic execution.

The fuzzer in Dr.PathFinder interacts with the DQN agent to determine the branch to search for during concolic execution. In this process, the DQN agent is trained using a gradient-based method, and the training can incur a large overhead even when hardware acceleration, such as a GPU, is supported. We implemented the DQN agent on a separate server to minimize the overhead. Therefore, when the fuzzer encounters a branch during the path search, it transmits the current basic block (state) to the DQN agent to obtain the action. Another advantage of this architecture is that multiple programs can be tested simultaneously using a single DQN agent. Such parallel testing allows the DQN agent to be free from overfitting. Dr.PathFinder is implemented with source code written in Python by adding and modifying hundreds of lines of Angr and Driller.

4.2 Dataset

Typically, the LAVA [13] and Cyber Grand Challenge (CGC) dataset [12] are used to evaluate the performance of fuzzers. Instead, we used the CB-multios dataset [50] because: 1) The LAVA dataset is a set of Linux-based binaries that were artificially injected with vulnerabilities. The binaries use various Linux system calls and library functions. Currently, Angr [44], which is the backend of Driller-AFL and Dr.PathFinder, does not model all functions in Linux. Therefore, LAVA is not a suitable choice for the evaluation. 2) The CGC was a challenge held by the Defense Advanced Research

Projects Agency (DARPA) to encourage the automated bug discovery. DARPA released the set of binaries used for challenge. The CGC dataset is beyond a simple laboratory-level dataset: it is a high-quality dataset with sufficient complexity as it approximates the functionality of real-world software or services. The CGC dataset is a popular dataset for evaluating the performance of fuzzers. However, it only works on the DARPA experimental Cyber Research Evaluation Environment used in the challenge. Therefore, CGC is not appropriate for evaluating our system.

The CB-multios dataset [50] is a migrated set of CGC binaries to Linux, Windows, and OS X environments. The CB-multios are provided in the form of compile scripts and source codes with vulnerabilities. We used `afl-clang` and `afl-clang++` to compile the binaries, and 32-bit binary files were generated after compiling.

We randomly selected 50 binaries from the CB-multios dataset for evaluation. Dr.PathFinder found more than 1 crash in 23 binaries within the given time. We leveraged 12 binaries, which Dr.PathFinder showed outstanding performance. For the rest of the binaries, Dr.PathFinder achieved a slightly superior performance compared to other fuzzers; however, some fuzzers performed better.

4.3 Experimental settings

All experiments were conducted in the same computing environment: Intel(R) Core(TM) i9-11900 2.50GHz, 32 GB RAM, and 64-bit Ubuntu 20.04 LTS. The deep learning model and the DQN agent server used an environment equipped with NVIDIA GTX 3080 Titan as the VGA for hardware acceleration. Because this computing environment is different from the initial Driller's runtime environment, the same experimental configuration in the research was not available. In our experiments, we set a single AFL process and five concolic execution processes. The fuzzer was run for 5 hours for each program, and "fuzz" was used as the initial seed.

In addition to the runtime environment, it is necessary to set hyperparameters to optimize the model adequately because Dr.PathFinder uses deep reinforcement learning. We configured the DQN agent as a stacked RNN-based model and set the initial parameters with "He"

initialization [20]. The Adam optimizer [25] was used to optimize the model parameters; the learning rate was set to 10^{-4} to train the model carefully. The replay memory capacity was set to 1 million, and a minibatch size of 100 ($m = 100$) was used. In addition, we configured the proportion of actions included in the minibatch to prevent the DQN agent from skewed training. When the number of experiences $(s_k, a_k, P_k, \Sigma_{k+1})$ in the replay memory exceeds 1 million, the oldest experiences are removed. Lastly, we set the immediate reward as 1 so that all actions are assessed equally as mentioned in Sect. 3.2.3. The reward setting is quite simple but enough to enable deeper path first search. For other hyper-parameters, we found the optimal values via experiments.

The performance of Dr.PathFinder depends on Q-network. Since Q-network is learned gradually, it is extremely difficult to evaluate the performance under the same conditions for each binary. Therefore, despite the advantage of running DQN on a separate server mentioned in Sect. 4.1, we initialize Q-network at the start of fuzzing for each binary. All binaries are tested under the same starting condition, and DQN gradually learns the state space on a binary-by-binary basis.

4.4 Performance evaluation depending on hyper-parameters

Most reinforcement learning models alternate between exploitation and exploration, learning the optimal policy. In order to effectively explore the execution paths of the given binary, Dr.PathFinder needs to properly be set ϵ , a hyper-parameter that controls the exploration rate in action selection. In addition, the DQN agent of Dr.PathFinder uses the discounted reward system to evaluate the action value. Policy learning is affected by the discount factor γ . Therefore, we observed the effects of ϵ and γ on performance of fuzzer through experiments and found optimal hyper-parameters.

Figure 3 shows the crash discovery progress according to ϵ and γ . Since it is not feasible to evaluate all possible hyper-parameter settings, in this experiment we set the scope of ϵ and γ to $\{0.1, 0.3, 0.5, 1.0\}$ and $\{0.3, 0.6, 0.9\}$ respectively. If ϵ is set 1.0, Dr.PathFinder always perform only exploration. Additionally, in order to evaluate the effects of the cell types constituting the Q-network, the Q-network was established with LSTM cells and simple RNN cells, respectively. Each setting is annotated in the legend in Fig. 3. In all experiments, the Q-network was initialized with the start of fuzzing and gradually trained as fuzzing progressed. In the meantime, the policy maintained stochastic policy².

As a result of the experiment, the highest performance was shown when the Q-network was composed of LSTM cells and ϵ and γ were set to 0.3 and 0.9, respectively (blue line). The lower the ϵ and γ , the lower the performance (gray and red lines, respectively). When ϵ is low, the opportunity to take a new action is less because the DQN agent mainly performs only exploitation. On the other hand, when only exploration was always performed ($\epsilon = 1.0$, turquoise line), the number of optimal actions in the given state decreased and the fuzzing performance was low. γ determines the ratio that reflects the sum of rewards expected to be received from the current state. In the experiment, Dr.PathFinder underperformed in settings with low γ (red line). This is because the DQN agent prioritized the present reward rather than the future reward and made a short-sighted action selection. Finally, in the case of Q-network configured with simple RNN cell (orange line) in the same hyper-parameter setting, it showed fairly good performance. Overall, the fuzzing performance was significantly different according to the same hyper-parameter setting. Also, with the same settings, when Q-network was built with LSTM cell rather than simple RNN cell, it showed better performance.

Figure 4 shows the cumulative depth of crashes found under each of the hyper-parameter settings shown in Fig. 3. To measure the depth of the crash, we used afl-showmap, one of AFL's toolkits. afl-showmap measures the transitions captured by instrumentation which was injected into the binary during the compilation and the number of times that transition was triggered. That is, adding up the number of times each transition, we can roughly get the length of the execution path. In Figs. 3 and 4, although not always, it seems that the discovery of new crashes is related to the length of the execution path. For example, in the case of Venture_Calculator or XStore, there has been a surge in the discovery of new crashes with the deep crashes. Also, according to the settings of ϵ and γ , the search depth was affected. In the case of KTY_Pretty_Printer or Rejistar, DQN agents with inadequately set ϵ and γ did not contribute to deep crash detection.

Based on the results of this experiment, we build the Q-network with LSTM cells. Also, for policy learning, ϵ and γ were set to 0.3 and 0.9, respectively.

² Some applications based on reinforcement learning models start learning with a stochastic policy, then they switch to a deterministic policy. However, in this experiment, Dr.PathFinder maintained a stochastic policy to continuously reflect new states and actions therein as fuzzing proceeds.

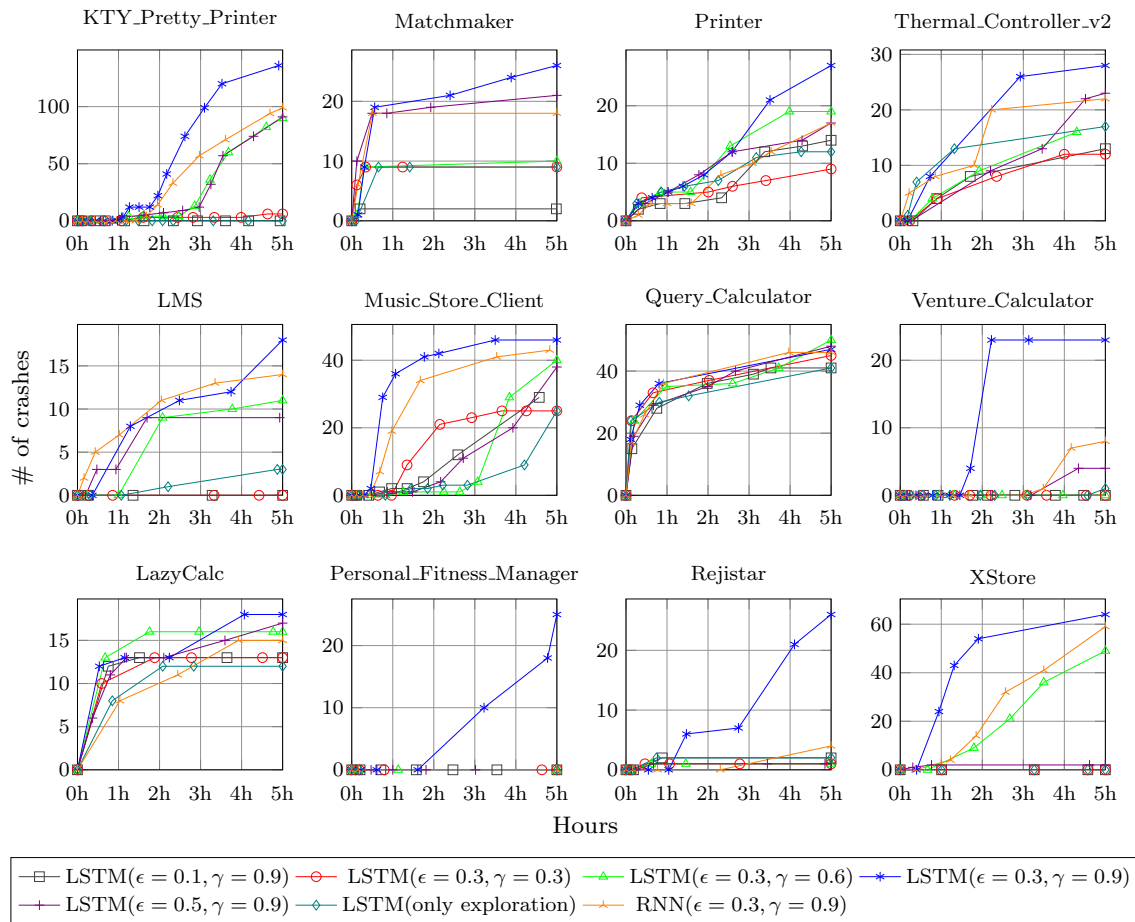


Fig. 3 Crash discovery progress according to hyper-parameters

4.5 Performance evaluation depending on the policy

Figure 5 shows the crash discovery performance of Dr.PathFinder depending on the policy types. Dr.PathFinder uses π -soft policy, a stochastic policy, to secure the diversity of experiences. In this experiment, we evaluated the performance of Dr.PathFinder under two policy settings. In both settings, ϵ and γ were set to 0.3 and 0.9, respectively, which showed the best performance in Sect. 4.4, and the Q-network was constructed with LSTM cells. The other configurations were the same as those used in Sect. 4.4. The blue line is the crash finding progress when the Q-network is continuously trained using π -soft policy as fuzzing progresses³. On the other hand, the red line is the crash discovery performance when Q-network is trained using π -soft policy for the first 2 hours and 30 minutes, then stopped learning and switched to deterministic policy. As shown in Fig. 5, for most binaries, both

configurations showed similar performance in the early stage of fuzzing following stochastic policy. However, in the later stage of the fuzzing, the configuration with deterministic policy performed noticeably worse. For example, in the case of KTY_Pretty_Printer or Thermal_Controller_v2, Dr.PathFinder does not detect any more crashes after policy change. As shown in Fig. 4, increasing the depth of the crashes means that Dr.PathFinder encounters new states as fuzzing progresses. However, as shown in the red line in Fig. 5, stopping the training the Q-network and selecting the action only greedily removes the opportunity for the Dr.PathFinder to learn the optimal action in the new state by referring only to the previous experience. Therefore, the experimental results in Fig. 5 imply that continuously learning new states and action selections in them contributes to the improvement of fuzzing performance.

³ In the DQN algorithm, the policy depends on the Q-network. Therefore, training the Q-network is equivalent to improving the policy.

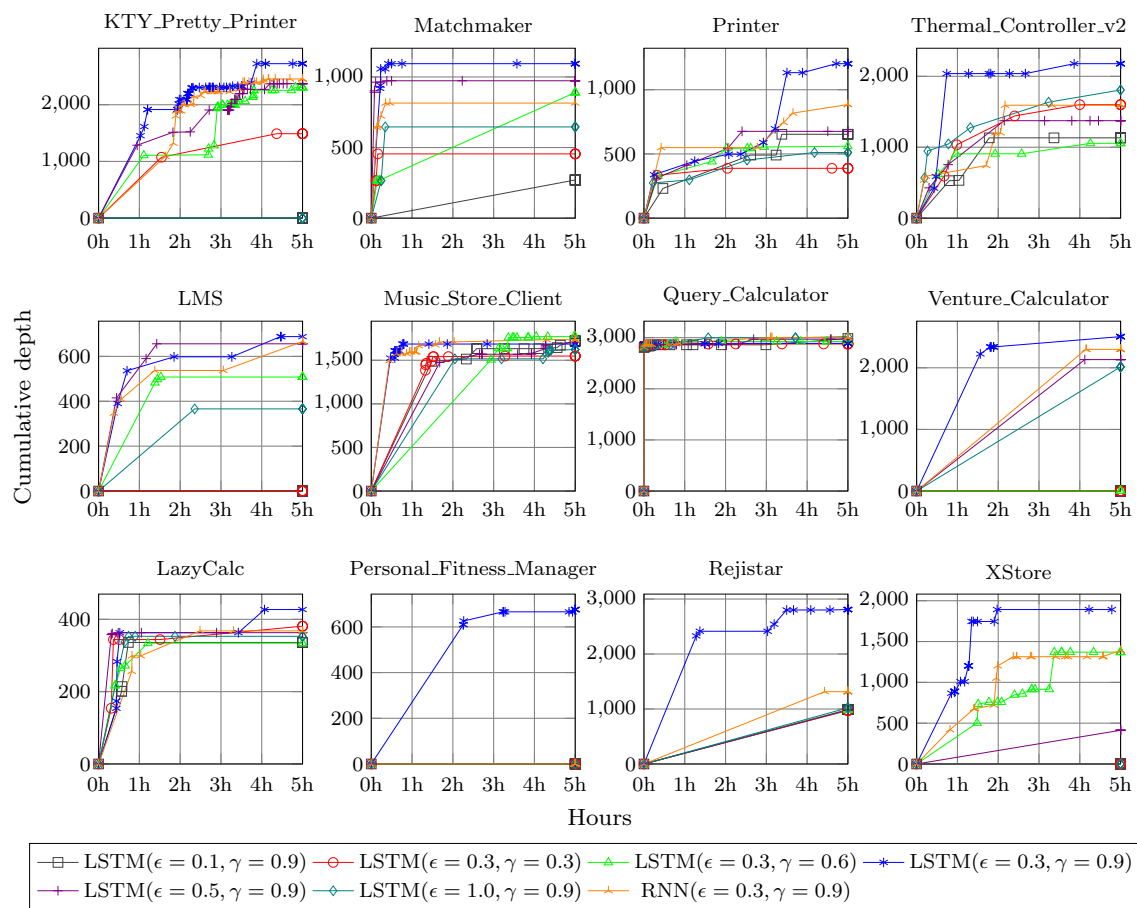


Fig. 4 Cumulative depth for discovered crashes between hyper-parameter settings

4.6 Comparison with other fuzzers for crash detection

In this section, we present experimental results comparing the crash discovery performance between Dr.PathFinder and other baseline fuzzers. In this experiment, the configuration of Dr.PathFinder uses a Q-network with LSTM cells, 0.3 for ϵ and 0.9 for γ as derived from Sects. 4.4 and 4.5. The policy of the DQN agent maintained the stochastic policy from the beginning to the end of fuzzing.

Figure 6 shows the progress of crash discovery among the fuzzers in the experimental settings. The x- and y-axes refer to the time and number of crashes, respectively. In Fig. 6, the blue line denotes the number of crashes found by Dr.PathFinder, red line denotes Driller-AFL, green line denotes AFL, purple line denotes AFL-fast, and the gray line denotes AFL-laf-intel (some invisible lines overlap with other lines). Table 2 lists the number of crashes deduplicated by afl-cmin (with option -C) of the AFL toolkits for the crashes observed in Fig. 6. Dr.PathFinder discovered 5.51 times more bugs than AFL and 1.84 times more bugs than Driller-AFL. As shown in Fig. 6, AFL and its family either do not discover any crashes or showed

limited performance compared to other fuzzers. This is because AFL did not find deep crashes due to the sanity check routine. In particular, because these binaries use the equality condition in the loop to determine the functionality to be executed, AFL relying on random mutation has limitations in effectively increasing the coverage. Compared to this, Dr.PathFinder and Driller-AFL continue to expand their test coverage, and even Dr.PathFinder finds more crashes than Driller-AFL in the same amount of time. To analyze the result, we measured the cumulative depth of the crashes discovered by both Dr.PathFinder and Driller-AFL using an afl-showmap and the results are shown in Fig. 7. As shown in Fig. 7, the crashes found by Dr.PathFinder was discovered in a similar or deeper path than Driller-AFL. Furthermore, the time of the crash matched the time of the increase in the path depth. For example, in the case of the Rejistar and LazyCalc, the point at which our method finds new crashes after the plateau state coincides with the point at which the path depth increases. Moreover, Dr.PathFinder performed better than Driller-AFL in terms of searching for deep paths.

Figure 8 shows the fuzzing results for binaries where Dr.PathFinder underperformed compared to Driller-AFL.

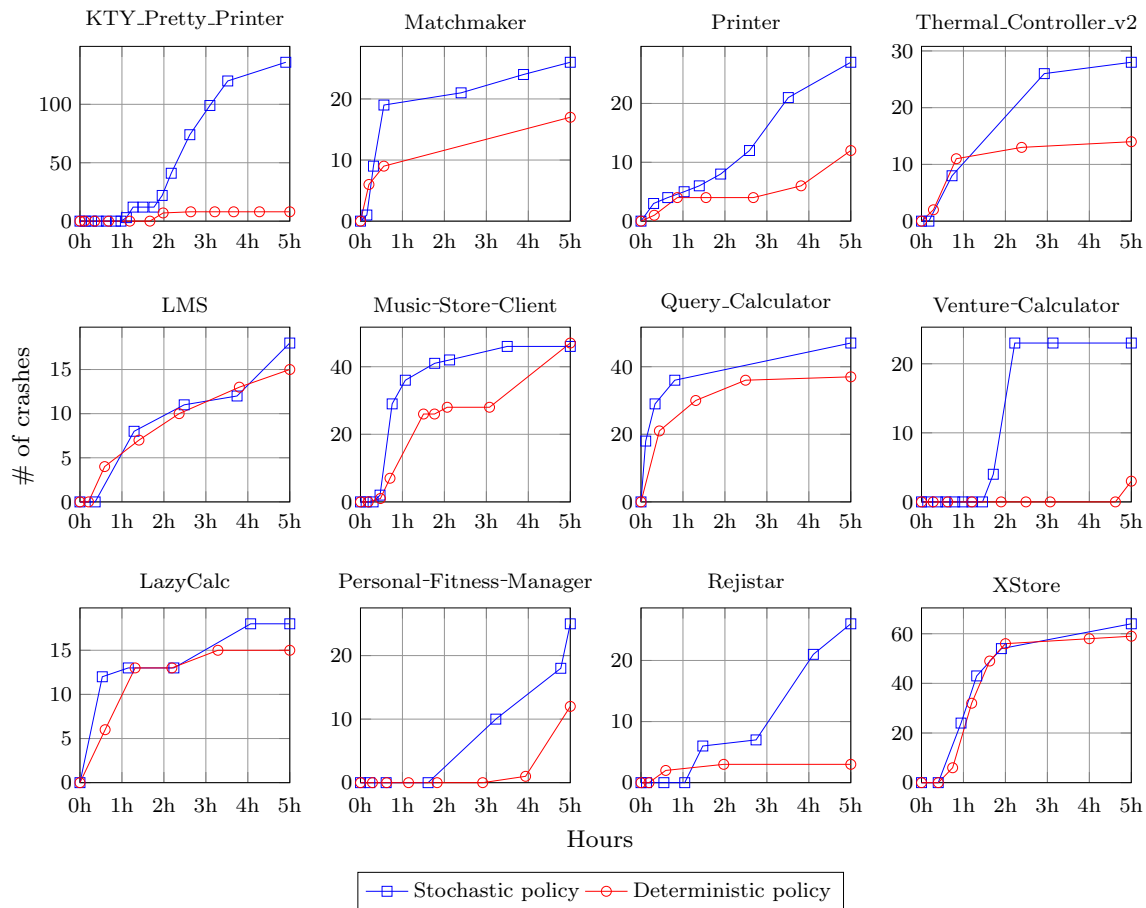


Fig. 5 Crash discovery progress according to policy types

For these binaries, Driller-AFL found additional crashes quickly after it found a crash. However, Dr.PathFinder found new crashes relatively slowly. As a result of the analysis, Driller-AFL found more shallow crashes than Dr.PathFinder. For example, all bugs found by Driller-AFL in *stack_vm* were located in a shallow execution path compared to those found by Dr.PathFinder. For the case that Dr.PathFinder found more bugs than Driller-AFL, although crashes were found deep, it does not mean that all crashes are located in the deep path. As shown in Fig. 8, if the bugs are located in a relatively shallow path, a full investigation strategy of Driller-AFL is more suitable. Dr.PathFinder has the lower performance to find bugs for programs operated by predefined commands. Generally, these programs process commands through loop statements. By nature, the more commands are processed in this process, the longer the length of the execution path is. The long path problem is severe when the exception handling for the wrong command is well implemented. As the Invalid commands make the execution path longer, it prevents the fuzzer from searching the bugs in the shallow paths. Dr.PathFinder leaves an opportunity for shallow

bugs to be explored by performing exploration stochastically but basically relies on AFL for shallow bugs.

4.7 Number of generated test cases

A deeper path-first search inevitably reduces the search space by pruning shallow paths, and therefore, the number of test cases generated by concolic execution decreases. Fig. 9 indicates the number of test cases created by the concolic execution of Dr.PathFinder and Driller-AFL on a log scale. The blue and red bars indicate the number of test cases generated by Dr.PathFinder and Driller-AFL, respectively. As shown in Fig. 9, the number of test cases from Dr.PathFinder was at least 2 times less than that of Driller-AFL. The results indicate that Dr.PathFinder outperforms Driller-AFL in crash discovery with a considerably smaller test case size.

Driller-AFL generates many test cases for relatively shallow paths by conducting concolic execution with a complete search. The test cases are then fed to the AFL. As a result, the AFL spends a considerable amount of time and resources testing a program using such test cases. Dr.PathFinder only provides test cases for deep paths to the

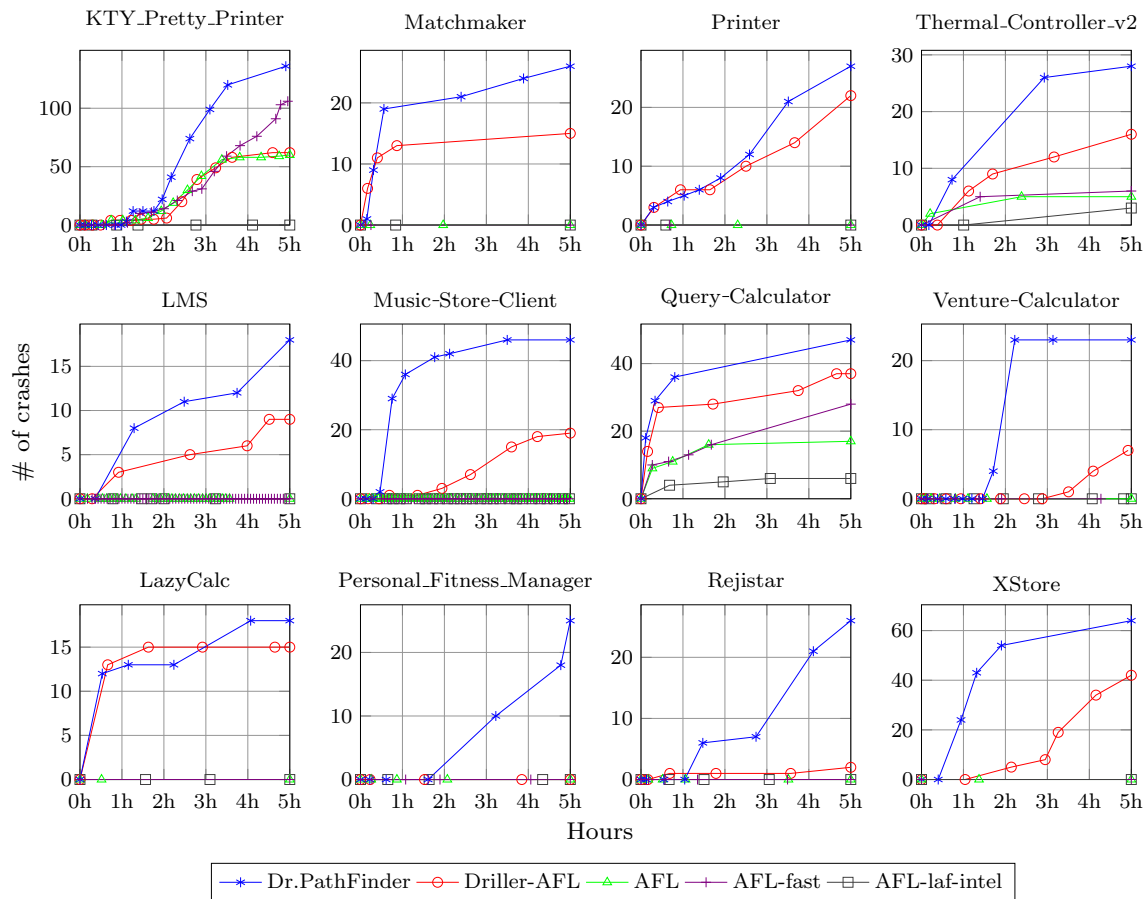


Fig. 6 Progress in the discovery of crashes between fuzzers

Table 2 Number of deduplicated crashes between fuzzers

Binary	Dr.PathFinder		Driller-AFL		AFL	
	Found	Dedup	Found	Dedup	Found	Dedup
Personal_Fitness_Manager	25	16	0	0	0	0
XStore	64	40	43	37	0	0
Thermal_Controller_v2	28	26	17	14	6	5
Venture_Calculator	23	20	8	7	0	0
LMS	18	8	10	7	0	0
Printer	27	21	23	16	0	0
Query_Calculator	47	34	38	19	18	14
Rejistar	26	23	3	2	0	0
KTY_Pretty_Printer	136	95	63	47	61	44
LazyCalc	18	15	16	12	0	0
Matchmaker	26	18	16	11	0	0
Music_Store_Client	46	31	20	17	0	0

“Found” and “dedup” refer to the number of discovered and deduplicated crashes, respectively

fuzzer, thereby allowing the fuzzer to focus on testing the deeper regions of the program.

4.8 Memory utilization

Figure 10 shows the memory utilization of Dr.PathFinder and Driller-AFL during fuzzing for the LMS binary and

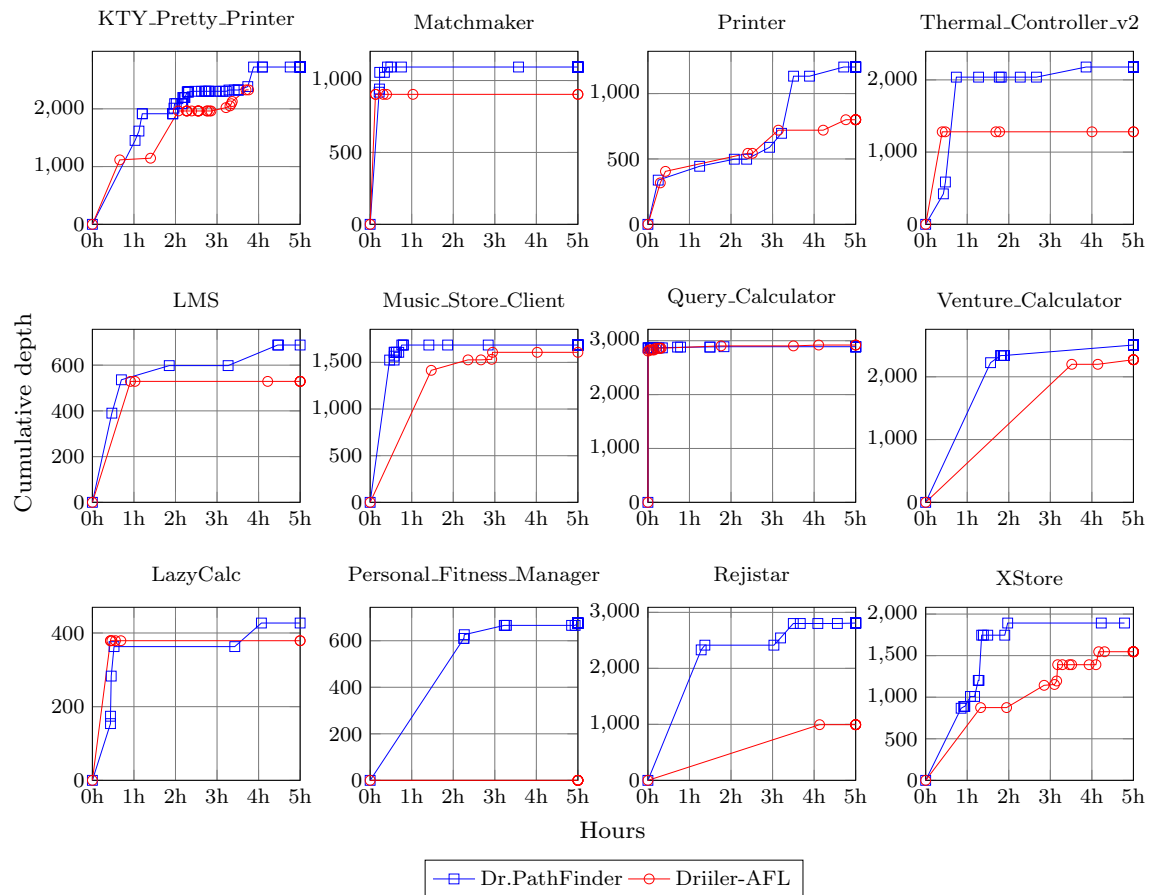
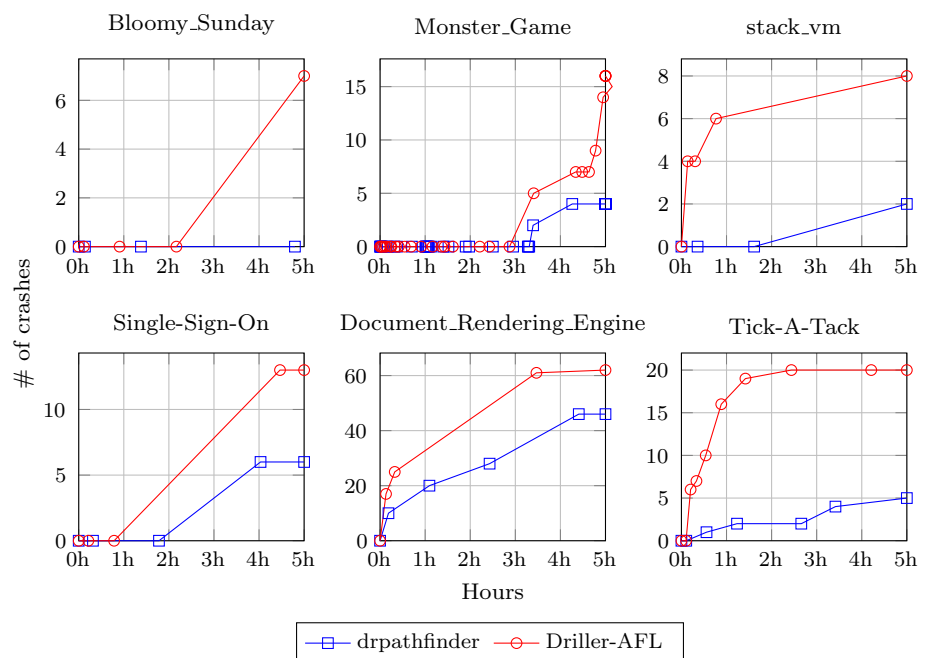


Fig. 7 Cumulative depth of discovered crashes between Dr.PathFinder and Driller-AFL

Fig. 8 The binaries and crash discovery progress where Driller-AFL outperformed



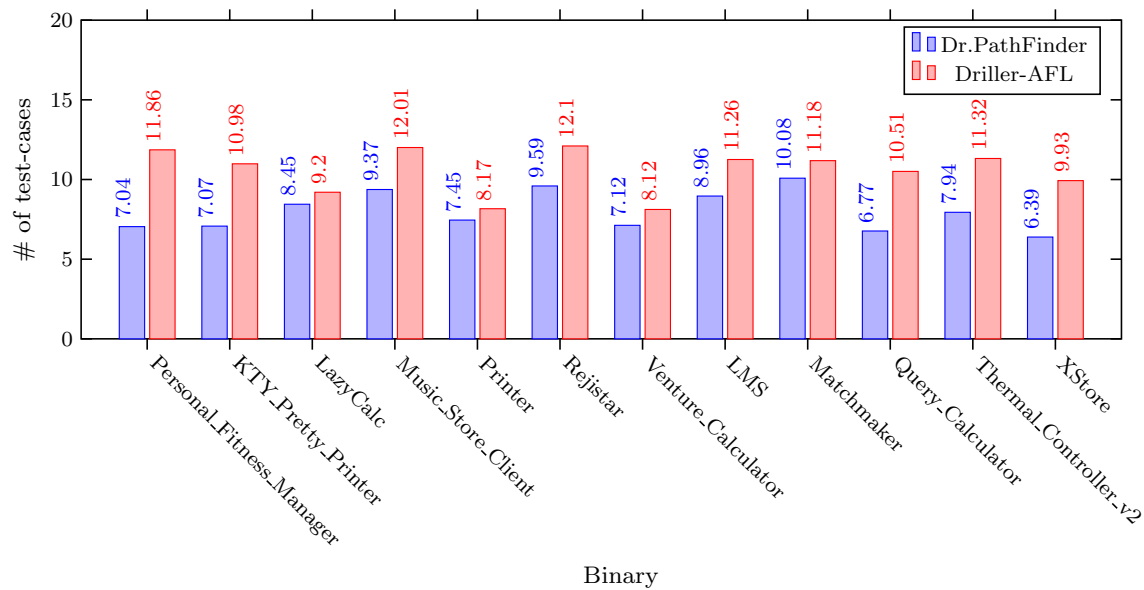


Fig. 9 Number of test cases generated by concolic execution

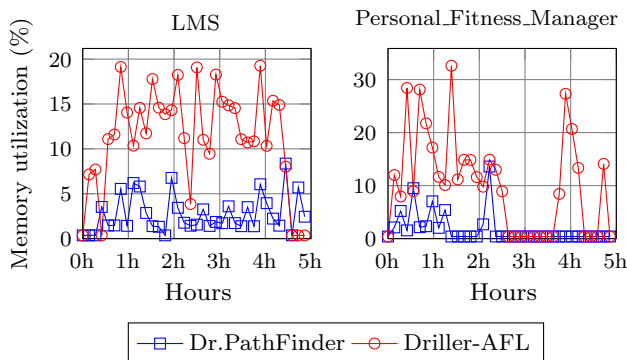


Fig. 10 Memory utilization between Dr.PathFinder and Driller-AFL

Personal_Fitness_Manager binary. We measured memory utilization using the process id (pid) of the concolic execution processes and AFL processes invoked by Dr.PathFinder for each binary. We selected LMS and Personal_Fitness_Manager because Driller-AFL generated the most test cases at these 2 binaries (LMS and Personal_Fitness_Manager) among the 12 binaries, which are appropriate for memory utilization comparison. The figure indicates the percentage of memory usage for both fuzzing and concolic execution on the total virtual memory space every 10 seconds. The x- and y-axes indicate the fuzzing time and memory utilization (%), respectively. From the experiment, for LMS, Dr.PathFinder and Driller-AFL showed 2.94 (± 2.36) % and 12.51 (± 6.5) % memory utilization on average and standard deviation, respectively. The Personal_Fitness_Manager showed 1.3 (± 2.05) % and 9.96 (± 8.79) % memory utilization, respectively. Dr.PathFinder used at least 8.66 % less memory than Driller-AFL on average, and the

deviation was also smaller than for Driller-AFL. This is because the deep reinforcement concolic execution of Dr.PathFinder does not trace the pruned paths, which leads to save memory. However, Driller-AFL showed less memory usage intermittently because it entered the idle state after using all test cases generated by AFL as seeds of concolic execution. Another interesting fact is that Dr.PathFinder used memory more efficiently than Driller-AFL. This is because unlike the scheduling of Driller-AFL, Dr.PathFinder stochastically conducted multiple concolic executions for a single test case.

```

1  int main (int secret_page_i, char*
    unused[]) {
2      ...
3      switch (cmd) {
4          case CMD_ADD: case CMD_SUB:
5          case CMD_MUL: case CMD_DIV:
6          case CMD_MOD:
7              // CMD_MOD task
8          case CMD_EXP:
9              // CMD_EXP task
10         case CMD_IMP:
11             // CMD_IMP task
12         case CMD_CMP:
13             // CMD_CMP task
14         case CMD_QUT:
15             // CMD_QUT task
16         default:
17             // default task
18     }
19     ...
20 }
21

```

Listing 1 Code snippet of LazyCalc

4.9 Case study for LazyCalc binary

In this section, we demonstrate a deeper path-first search by the DQN agent via a case study. The binary used in this experiment was `LazyCalc` from `CB-multios`. `LazyCalc` provides basic binary operations, such as addition, subtraction, multiplication, division, and modulo. List 1 shows a part of the core source code of `LazyCalc`. This binary executes the operation corresponding to the value of the `cmd` variable using the switch-case statement. Figure 11 shows a simplified control-flow graph and guided-concolic execution by the DQN agent for List 1. In the figure, the blue and red lines indicate branches where the condition is satisfied (true branch) and the branch where the condition is unsatisfied (false branch), respectively. The black dashed line indicates the action of the DQN agent. The Q-function value for each action is given above the black dashed line $[Q_\theta(s_t, \text{tb}), Q_\theta(s_t, \text{tt}), Q_\theta(s_t, \text{tf})] \rightarrow \text{action}$. In addition, in this experiment, the DQN agent did not conduct the exploration randomly by selecting an action but only conducted exploitation following the *maximum greedy policy*. The branches in bold red arrows represent those selected by the agent. The agent and Q-function defined in Sect. 3 imply a path with more branches as the longer path. Based on this definition, the path with five branches is the longest. In Fig. 11, the agent guided the concolic execution to search for the longest path first by selecting only the false branch continuously from the top basic block, which is the start of the switch-case statement. In addition, the paths created by the true branch pruned in this process are no longer traced; instead, they are saved. Subsequently, the

SMT solver generated test cases that enabled the paths. These test cases were used to search for the pruned path in the next concolic execution.

5 Limitations

Despite the impressive experimental results, `Dr.PathFinder` has several limitations:

- Deep reinforcement concolic execution has the weakness of ignoring bugs located in shallow paths because the design leads to a deep path. This limits the ability of `Dr.PathFinder` to discover bugs. `Dr.PathFinder` basically relies on `AFL` for finding shallow bugs. Although many studies state that fuzzers like `AFL` are specialized for finding shallow bugs [23, 30, 31], this is still not enough to find the bugs in the shallow paths. `Dr.PathFinder` controls the search depth through a reward system by nature. In Sect. 3.2, we configured the reward that the DQN agent takes for an action to a positive number; therefore, the longer the path, the larger the accumulated reward. An obvious approach to search for shallow paths is to set the reward to a negative value. However, paths searched by such a configuration may be shorter than necessary, and therefore, the functionalities of the program cannot be executed with early termination.
- Paths that can be deeper than the traced paths may be included in paths pruned by the DQN agent. In particular, this problem worsens for programs with a very wide state space. Branches leading to deeper paths

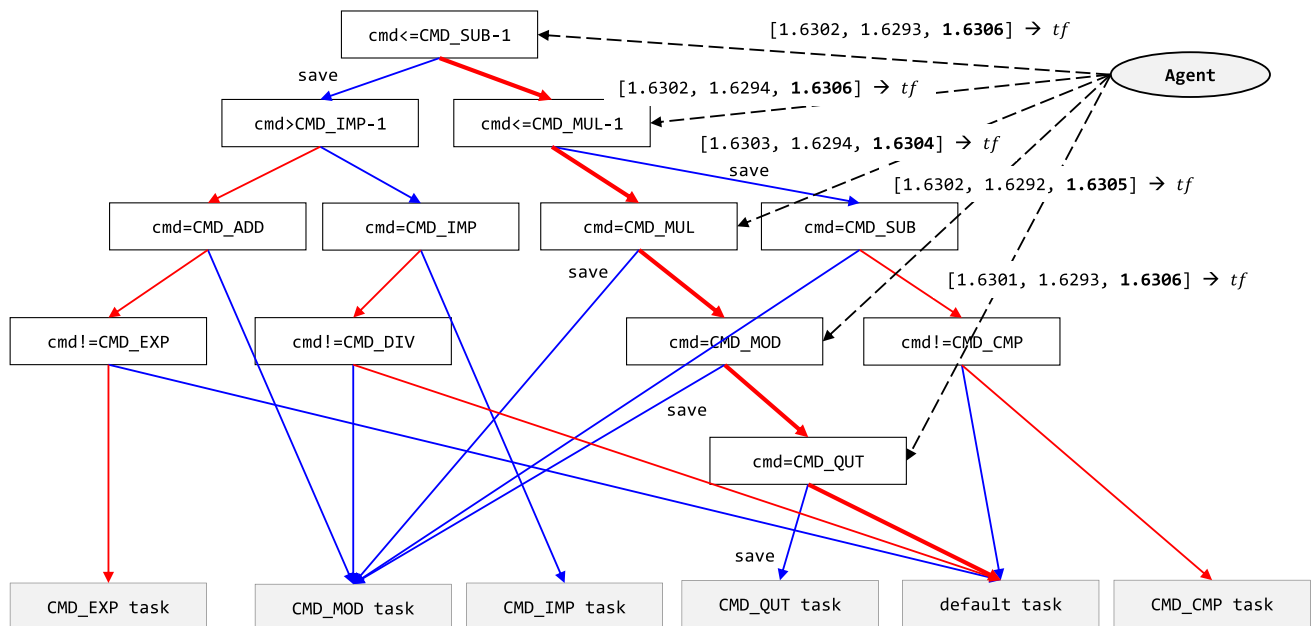


Fig. 11 The control-flow graph for Listing 1

appear early in program execution and determine the functionality to be executed; thus, bad branch selection can prevent the fuzzer from testing some functionalities.

- Dr.PathFinder uses a scheduler designed to allow the stochastic evaluation of the pruned paths. However, because the rate of exploration in the ϵ -soft policy of the DQN agent is lower than that of the exploitation, it is unlikely that the pruned paths in the previous concolic execution will be searched.
- Last but not least, Dr.PathFinder's objective is only designed to preferentially follow paths that are expected to lead to the deeper areas of the program during execution. This suggests that, in other words, the bug discovery performance of Dr.PathFinder is not directly reflected in the objective. A well-trained Q-network searches just deep paths regardless of bugs under the deterministic policy. This can limit the bug finding performance of Dr.PathFinder. Unlike the main streams of reinforcement learning, it is required to continuously employ the stochastic policy to randomly select the action based on the action value and to find new bugs. This requirement makes it difficult to converge the learning of Q-network to the optimal policy. These weaknesses of Dr.PathFinder are very interesting and enough to inspire new studies.

6 Conclusion

A deep reinforcement concolic execution algorithm that first searches deep paths using reinforcement learning and hybrid fuzzing solution is proposed in this paper. The proposed DQN-based reinforcement learning agent observes the basic block as a state during concolic execution, selects an action, and traces the branch where a potentially longer path is expected. The DQN agent receives rewards for the performed actions and learns the optimal policy. In addition, we presented Dr.PathFinder, which implemented the proposed algorithm and observed promising results via comparison with state-of-the-art fuzzers on the benchmark dataset. The experimental results indicated that Dr.PathFinder achieved the same or better bug discovery performance as existing fuzzers for deep bugs; finally, it explored the deep region of the program at an early stage. Dr.PathFinder generated fewer test cases than the existing hybrid fuzzer and efficiently used the memory.

In addition to these experimental results, we discuss several limitations of Dr.PathFinder. In the future, we plan to conduct a study that directly links bugs with the reward

system, as well as a way to balance the shallow and deep paths.

Acknowledgements This research was supported by a grant from the Korea Health Technology R&D Project through the Korea Health Industry Development Institute (KHIDI) and funded by the Ministry of Health & Welfare, Republic of Korea (Grant Number: HI19C0791).

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Aschermann C, Schumilo S, Blazytko T, Gawlik R, Holz T (2019) REDQUEEN: fuzzing with input-to-state correspondence. In: Proceedings of NDSS, pp 1–15. <https://doi.org/10.14722/ndss.2019.23371>
2. Barreto A, Dabney W, Munos R, Hunt JJ, Schaul T, van Hasselt H, Silver D (2018) Successor features for transfer in reinforcement learning
3. Barrett C, Stump A, Tinelli C (2010) The SMT-LIB standard: version 2.0. In: Gupta A, Kroening D (eds.) Proceedings of international work satisfy modul theory, p 14
4. Böhme M, Pham VT, Roychoudhury A (2016) Coverage-based greybox fuzzing as Markov chain. Proc ACM Conf Comput Commun Secur. <https://doi.org/10.1145/2976749.2978428>
5. Bottou L (2012) Stochastic gradient descent tricks. Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics). <https://doi.org/10.1007/978-3-642-35289-8-25>
6. Brumley D, Jager I, Avgerinos T, Schwartz EJ (2011) BAP: A binary analysis platform. Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics) pp. 464–469. https://doi.org/10.1007/978-3-642-22110-1_37
7. Cadar C, Dunbar D, Engler D (2019) Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In: Proc USENIX Symp Oper Syst Des Impl, pp 209–224
8. Cha SK, Avgerinos T, Rebert A, Brumley D (2012) Unleashing Mayhem on binary code. In: Proc IEEE Symp Secur Priv, pp 380–394. <https://doi.org/10.1109/SP.2012.31>
9. Chen P, Chen H (2018) Angora: efficient fuzzing by principled search. In: Proceedings IEEE symposium on security and privacy, pp. 711–725. <https://doi.org/10.1109/SP.2018.00046>
10. Chipounov V, Kuznetsov V, Candea G (2011) S2E: a platform for in-vivo multi-path analysis of software systems. In: Proceedings international conference on architectural support for programming languages and operating systems, pp. 265–278. <https://doi.org/10.1145/1950365.1950396>
11. De Moura L, Bjørner N (2008) Z3: an efficient SMT solver. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) pp. 337–340. https://doi.org/10.1007/978-3-540-78800-3_24
12. Defence Advanced Research Projects Agency (DARPA): Cyber Grand Challenge (CGC) (2016). <https://www.darpa.mil/program/cyber-grand-challenge>
13. Dolan-Gavitt B, Hulin P, Kirda E, Leek T, Mambretti A, Robertson W, Ulrich F, Whelan R (2016) LAVA: large-scale automated vulnerability addition. In: Proceedings IEEE

- Symposium on Security and Privacy, pp. 110–121. <https://doi.org/10.1109/SP.2016.15>
14. Enck W, Gilbert P, Han S, Tendulkar V, Chun BG, Cox LP, Jung J, McDaniel P, Sheth AN (2014) TaintDroid: an information-flow tracking system for realtime privacy monitoring on smartphones. *ACM Trans Comput Syst*. <https://doi.org/10.1145/2619091>
 15. Ganai M, Lee D, Gupta A (2012) DTAM: dynamic taint analysis of multi-threaded programs for relevancy. *Proc ACM SIGSOFT Int Symp Found Softw Eng*. <https://doi.org/10.1145/2393596.2393650>
 16. Ganesh V, Leek T, Rinard M (2009) Taint-based directed whitebox fuzzing. *Proc Int Conf Softw Eng*. <https://doi.org/10.1109/ICSE.2009.5070546>
 17. Godefroid P, Klarlund N, Sen K (2005) DART: directed automated random testing. *ACM SIGPLAN Not* 1:2. <https://doi.org/10.1145/1064978.1065036>
 18. Google: Honggfuzz (2016). <https://github.com/google/honggfuzz>
 19. Haller I, Slowinska A, Neugschwandtner M, Bos H (2013) Dowsing for overflows: a guided fuzzer to find buffer boundary violations. In: *Proceedings of USENIX security symposium*, pp 49–64
 20. He K, Zhang X, Ren S, Sun J (2015) Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In: *Proceedings IEEE international conference on computer vision*, pp. 1026–1034. <https://doi.org/10.1109/ICCV.2015.123>
 21. Hochreiter S, Schmidhuber J (1997) Long short-term memory. *Neural Comput* 1:2. <https://doi.org/10.1162/neco.1997.9.8.1735>
 22. Ioffe S, Szegedy C (2015) Batch normalization: accelerating deep network training by reducing internal covariate shift. In: *Proceedings of international conference on machine learning*, pp. 448–456
 23. Ispoglou KK, Austin D, Mohan V, Payer M (2020) FuzzGen: automatic fuzzer generation. In: *Proceedings of 29th USENIX security symposium*
 24. Kim S, Faerevaag M, Jung M, Jung S, Oh D, Lee J, Cha SK (2017) Testing intermediate representations for binary analysis. In: *Proceedings of IEEE/ACM international conference on automated software engineering*, pp 353–364. <https://doi.org/10.1109/ASE.2017.8115648>
 25. Kingma DP, Ba J (2017) Adam: a method for stochastic optimization
 26. Laf-Intel: circumventing fuzzing roadblocks with compiler transformations (2016). <https://lafintel.wordpress.com/2016/08/15/circumventing-fuzzing-roadblocks-with-compiler-transformations/>
 27. Landi W (1992) Undecidability of static analysis. *ACM Lett Program Lang Syst*. <https://doi.org/10.1145/161494.161501>
 28. Lattner C, Adve V (2004) LLVM: a compilation framework for lifelong program analysis & transformation. In: *International symposium on code generation and optimization*, pp 75–86. <https://doi.org/10.1109/CGO.2004.1281665>
 29. Lecun Y, Bengio Y, Hinton G (2015) Deep learning. <https://doi.org/10.1038/nature14539>
 30. Liang J, Jiang Y, Wang M, Jiao X, Chen Y, Song H, Choo KKR (2020) DeepFuzzer: accelerated deep Greybox fuzzing. *IEEE Trans Depend Secur Comput*. <https://doi.org/10.1109/TDSC.2019.2961339>
 31. Liang J, Wang M, Chen Y, Jiang Y, Zhang R (2018) Fuzz testing in practice: obstacles and solutions. In: *25th IEEE international conference software analysis evolution reengineering, SANER 2018—Proceedings, vol 2018-March*. <https://doi.org/10.1109/SANER.2018.8330260>
 32. Lin LJ (1993) Reinforcement learning for robots using neural networks. *Carnegie-Mellon Univ Pittsburgh PA School of Computer Science, Tech. rep*
 33. Luk CK, Cohn R, Muth R, Patil H, Klauser A, Lowney G, Wallace S, Reddi VJ, Hazelwood K (2005) Pin: Building customized program analysis tools with dynamic instrumentation. In: *Proceedings of ACM SIGPLAN Conference programming language implementation*, pp. 190–200
 34. Lyu C, Ji S, Zhang C, Li Y, Lee WH, Song Y, Beyah R (2019) MOPT: optimized mutation scheduling for fuzzers. In: *Proceedings of USENIX security symposium*, pp 1949–1966
 35. Masri W, Podgurski A, Leon D (2004) Detecting and debugging insecure information flows. In: *Proceedings—international symposium on software reliability engineering*, pp 198–209. <https://doi.org/10.1109/issre.2004.17>
 36. Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, Graves A, Riedmiller M, Fidjeland AK, Ostrovski G, Petersen S, Beattie C, Sadik A, Antonoglou I, King H, Kumaran D, Wierstra D, Legg S, Hassabis D (2015) Human-level control through deep reinforcement learning. *Nature*. <https://doi.org/10.1038/nature14236>
 37. Nethercote N, Seward J (2007) Valgrind: A framework for heavyweight dynamic binary instrumentation. In: *Proceedings ACM SIGPLAN conference on programming language design and implementation*, pp 89–100. <https://doi.org/10.1145/1250734.1250746>
 38. Peng H, Shoshitaishvili Y, Payer M (2018) T-Fuzz: fuzzing by program transformation. In: *Proceedings of IEEE symposium on security and privacy*, pp 697–710. <https://doi.org/10.1109/SP.2018.00056>
 39. Rawat S, Jain V, Kumar A, Cojocar L, Giuffrida C, Bos H (2017) VUzzer: application-aware evolutionary fuzzing. In: *Proceedings of NDSS*, pp 1–14. <https://doi.org/10.14722/ndss.2017.23404>
 40. Rummery GA, Niranjan M (1994) On-line Q-learning using connectionist systems, vol 37. Cambridge University, Engineering Department
 41. Schwartz EJ, Avgerinos T, Brumley D (2010) All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask). In: *Proceedings of IEEE symposium on security and privacy*, pp 317–331. <https://doi.org/10.1109/SP.2010.26>
 42. Sen K, Marinov D, Agha G (2005) CUTE: a concolic unit testing engine for C. In: *Proceedings of European Software Engineering Conference held jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software*, pp 263–272
 43. Sengupta S, Basak S, Peters RA (2018) Particle Swarm Optimization: a survey of historical and recent developments with hybridization perspectives. <https://doi.org/10.3390/make1010010>
 44. Shoshitaishvili Y, Wang R, Salls C, Stephens N, Polino M, Dutcher A, Grosen J, Feng S, Hauser C, Kruegel C, Vigna, G (2016) SOK: (State of) the art of war: offensive techniques in binary analysis. In: *Proceedings of IEEE symposium on security and privacy*, pp 138–157. <https://doi.org/10.1109/SP.2016.17>
 45. Silver D, Huang A, Maddison CJ, Guez A, Sifre L, Van Den Driessche G, Schrittwieser J, Antonoglou I, Panneershelvam V, Lanctot M, Dieleman S, Grewe D, Nham J, Kalchbrenner N, Sutskever I, Lillicrap T, Leach M, Kavukcuoglu K, Graepel T, Hassabis D (2016) Mastering the game of Go with deep neural networks and tree search. *Nature*. <https://doi.org/10.1038/nature16961>
 46. Silver D, Schrittwieser J, Simonyan K, Antonoglou I, Huang A, Guez A, Hubert T, Baker L, Lai M, Bolton A, Chen Y, Lillicrap T, Hui F, Sifre L, Van Den Driessche G, Graepel T, Hassabis D (2017) Mastering the game of Go without human knowledge. *Nature*. <https://doi.org/10.1038/nature24270>
 47. Stephens N, Grosen J, Salls C, Dutcher A, Wang R, Corbetta J, Shoshitaishvili Y, Kruegel C, Vigna G (2017) Driller: augmenting fuzzing through selective symbolic execution. In:

- Proceedings of NDSS, pp 1–16. <https://doi.org/10.14722/ndss.2016.23368>
48. Sun M, Wei T, Lui JC (2016) TaintART: a practical multi-level information-flow tracking system for Android RunTime. In: Proceedings of ACM conference computer communications security, pp 331–342. <https://doi.org/10.1145/2976749.2978343>
 49. Sutton RS, Barto AG (2018) Reinforcement learning: an introduction, 2nd edn, vol 3. MIT Press
 50. Trail of Bits: DARPA challenges sets for Linux, Windows, and macOS (2016). <https://github.com/trailofbits/cb-multios>
 51. Mnih V, Kavukcuoglu K, Silver D, Graves A, Antonoglou I, Daan W, Riedmiller M (2016) Playing atari with deep reinforcement learning. <https://doi.org/10.1038/nature14236>
 52. Wang T, Wei T, Gu G, Zou W (2010) TaintScope: a checksum-aware directed fuzzing tool for automatic software vulnerability detection. In: Proceedings of IEEE symposium on security and privacy, pp 497–512. <https://doi.org/10.1109/SP.2010.37>
 53. Watkins CJCH, Dayan P (1992) Q-learning. Mach Learn. <https://doi.org/10.1007/bf00992698>
 54. Wei CY, Hong YT, Lu CJ (2017) Online reinforcement learning in stochastic games
 55. Yun I, Lee S, Xu M, Jang Y, Kim T (2018) QSYM: a practical concolic execution engine tailored for hybrid fuzzing. In: Proceedings of USENIX security symposium, pp 745–761
 56. Zakeri Nasrabadi M, Parsa S, Kalaei A (2021) Format-aware learn & fuzz: deep test data generation for efficient fuzzing. Neural Comput Appl 33(5):1–17. <https://doi.org/10.1007/s00521-020-05039-7>
 57. Zalewski M (2017) American fuzzy lop. <https://lcamtuf.coredump.cx/afl/>
 58. Zhao L, Duan Y, Yin H, Xuan J (2019) Send hardest problems my way: probabilistic path prioritization for hybrid fuzzing. In: Proceedings of NDSS, pp 1–15. <https://doi.org/10.14722/ndss.2019.23504>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.